

SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Faculty of Computing Department of Information Technology

Web-Based Driving License Issuing Service

DRIVELINK

Assignment Part - 02

Relational Schema Design, SQL Implementation, Queries & Advanced SQL Features

Group ID - 026-Y2-S1-MLB-WEB1G2-03

IT Number	Name
IT25100815	Simra M.A.F
IT25100817	Weerasinghe W.P.D.V
IT25100821	Fernando U.A.E.N
IT25100830	Thasalogithan S
IT25100851	Madampage K.S
IT25100856	Karunarathne D.K.K.P

IT2140 - Database Design and Development | Year 2, Semester 1 - 2026

Table of Contents

Table of Contents **2**

Part A - Mapping EER to Relational Schema **3**

A.1 ISA Mapping Strategy 3

A.2 Special EER Features ---> Mapping 3

A.3 Relational Schema (standard PK / FK notation) 4

A.4 Schema Refinement Notes 6

Part B - SQL DDL Implementation **7**

Part C - Insert Sample Data **14**

Part D - SQL Queries and Outputs **23**

Query 1 - Simple SELECT 23

Query 2 - JOIN 23

Query 3 - Aggregation 24

Query 4 - GROUP BY / HAVING 24

Query 5 - Subquery 25

Query 6 - Complex multi-table JOIN 25

Part E - Stored Procedures **26**

IT25100815 - Simra M.A.F 26

IT25100817 - Weerasinghe W.P.D.V 27

IT25100821 - Fernando U.A.E.N 28

IT25100830 - Thasalogithan S 30

IT25100851 - Madampage K.S 33

IT25100856 - Karunarathne D.K.K.P 34

Part F - Trigger Implementation **36**

Trigger 1 - trg_auto_notify_exam_result 36

Trigger 2 - trg_check_exam_slot_capacity 38

EER corrected diagram **40**

Part A - Mapping EER to Relational Schema

A.1 ISA Mapping Strategy

The EER diagram has two disjoint (d) ISA hierarchies: **USER ---> 7 subtypes (APPLICANT, REGISTRATION_OFFICER, TEST_COORDINATOR, DRIVING_EXAMINER, DRIVING_TRAINER, MEDICAL_OFFICER, SYSTEM_ADMIN), and APPLICANT ---> EXISTING_LICENSE_HOLDER.**

Chosen strategy: Separate table per subtype. Each subtype table reuses the parent's primary key as its own PK and FK.

- Subtypes have distinct attributes - combining them would create many NULLs.
- The disjoint (d) constraint is naturally enforced: a user can only have a row in one subtype table.
- Queries on a single role stay efficient, touching only the relevant table.
- Referential integrity between parent and subtype is enforced by the FK constraint.

A.2 Special EER Features ---> Mapping

EER Feature	Mapping
Composite attr. Name	Flattened to FirstName, LastName in USERS
Composite attr. Address	Flattened to Street, City, Province, PostalCode in USERS
Multivalued {Phone}	USER_PHONE(UserID, Phone)
Multivalued {VehicleCate ries}	LICENSE_VEHICLE_CATE RY(LicenseID, VehicleCate ry)
Recursive: Examiner supervises Examiner	Self-referencing FK SupervisorID in DRIVING_EXAMINER
Aggregation: Exam + Trial ---> Application	Checked together inside ProcessLicenseApproval (Part E)
M:N Exam - Slot	EXAM_SLOT_ASSIGNMENT(ExamID, SlotID)
M:N Trainer - Applicant (+AssignDate)	TRAINER_APPLICANT(TrainerID, ApplicantID, AssignDate)
M:N Applicant - Session	APPLICANT_SESSION(ApplicantID, SessionID)
1:1 Appointment - Test	UNIQUE(AppointmentID) in MEDICAL_TEST
1:1 Applicant - License	UNIQUE(ApplicantID) in DRIVING_LICENSE

A.3 Relational Schema (standard PK / FK notation)

Primary keys are underlined; foreign keys are listed with an arrow to the table and column they reference - the same convention used for the Player / Match / Play example.

1. **USERS** (**UserID**, FirstName, LastName, Email, Password, Role, Street, City, Province, PostalCode)

   Parent table of the ISA hierarchy (7 disjoint subtypes)

2. **USER_PHONE** (**UserID**, **Phone**)

   FK UserID --> USERS(UserID)

   Multivalued attribute {Phone} of USER

3. **APPLICANT** (**UserID**, NIC, DOB, BloodGroup)

   FK UserID --> USERS(UserID)

   ISA subtype of USERS

4. **EXISTING_LICENSE_HOLDER** (**UserID**, LicenseNo, ExpiryDate)

   FK UserID --> APPLICANT(UserID)

   ISA subtype of APPLICANT

5. **REGISTRATION_OFFICER** (**UserID**, OfficerID, Department)

   FK UserID --> USERS(UserID)

6. **TEST_COORDINATOR** (**UserID**, CoordID)

   FK UserID --> USERS(UserID)

7. **DRIVING_EXAMINER** (**UserID**, ExaminerID, Qualification, *SupervisorID*)

   FK UserID --> USERS(UserID) | SupervisorID ---> DRIVING_EXAMINER(UserID)

   SupervisorID = recursive "supervises" relationship

8. **DRIVING_TRAINER** (**UserID**, TrainerID, Specialization)

   FK UserID --> USERS(UserID)

9. **MEDICAL_OFFICER** (**UserID**, MedOfficerID, Designation)

   FK UserID --> USERS(UserID)

10. **SYSTEM_ADMIN** (**UserID**, AdminID)

    FK UserID --> USERS(UserID)

11. **LICENSE_APPLICATION** (**ApplicationID**, *ApplicantID*, *OfficerID*, SubmitDate, Status, LicenseCate ry)
    FK ApplicantID --> APPLICANT(UserID) | OfficerID ---> REGISTRATION_OFFICER(UserID)

12. **DOCUMENT** (**DocumentID**, *ApplicationID*, Type, FilePath, UploadDate, Status)
    FK ApplicationID --> LICENSE_APPLICATION(ApplicationID)

13. **MEDICAL_APPOINTMENT** (**AppointmentID**, *ApplicantID*, *MedOfficerID*, AppointmentDate, AppointmentTime, Status)
    FK ApplicantID --> APPLICANT(UserID) | MedOfficerID ---> MEDICAL_OFFICER(UserID)

14. **MEDICAL_TEST** (**MedTestID**, *AppointmentID (UQ)*, TestDate, Result, Status)
    FK AppointmentID --> MEDICAL_APPOINTMENT(AppointmentID)
    UNIQUE(AppointmentID) enforces 1:1

15. **EXAM_SLOT** (**SlotID**, *CoordID*, SlotDate, SlotTime, Location, Capacity)
    FK CoordID --> TEST_COORDINATOR(UserID)

16. **DRIVING_EXAM** (**ExamID**, *ApplicantID*, *ExaminerID*, ExamDate, Location, Type, Score, Status)
    FK ApplicantID --> APPLICANT(UserID) | ExaminerID ---> DRIVING_EXAMINER(UserID)

17. **EXAM_SLOT_ASSIGNMENT** (**ExamID**, **SlotID**)
    FK ExamID --> DRIVING_EXAM(ExamID) | SlotID ---> EXAM_SLOT(SlotID)
    Junction table for M:N assignment

18. **DRIVING_TRIAL** (**TrialID**, *ApplicantID*, *ExaminerID*, TrialDate, Result, Remarks, Status)
    FK ApplicantID --> APPLICANT(UserID) | ExaminerID ---> DRIVING_EXAMINER(UserID)

19. **TRAINING_SESSION** (**SessionID**, *TrainerID*, SessionDate, Progress, Remarks)
    FK TrainerID --> DRIVING_TRAINER(UserID)

20. **TRAINER_APPLICANT** (**TrainerID**, **ApplicantID**, AssignDate)
    FK TrainerID --> DRIVING_TRAINER(UserID) | ApplicantID ---> APPLICANT(UserID)
    Junction table for M:N; AssignDate is the relationship attribute

21. **APPLICANT_SESSION** (**ApplicantID**, **SessionID**)

FK ApplicantID --> APPLICANT(UserID) | SessionID ---> TRAINING_SESSION(SessionID)

Junction table for M:N

---

22. **DRIVING_LICENSE** (**LicenseID**, *ApplicantID (UQ)*, *OfficerID*, IssueDate, ExpiryDate, Type)

FK ApplicantID --> APPLICANT(UserID) | OfficerID ---> REGISTRATION_OFFICER(UserID)

UNIQUE(ApplicantID) enforces 1:1

---

23. **LICENSE_VEHICLE_CATE RY** (**LicenseID**, **VehicleCate ry**)

FK LicenseID --> DRIVING_LICENSE(LicenseID)

Multivalued attribute {VehicleCate ries}

---

24. **RENEWAL_REQUEST** (**RenewalID**, *LicenseHolderID*, *OfficerID*, RequestDate, Status)

FK LicenseHolderID --> EXISTING_LICENSE_HOLDER(UserID) | OfficerID ---> REGISTRATION_OFFICER(UserID)

---

25. **PAYMENT** (**PaymentID**, *ApplicantID*, Amount, PaymentDate, Method, Status, Type)

FK ApplicantID --> APPLICANT(UserID)

---

26. **NOTIFICATION** (**NotifID**, *ApplicantID*, Message, NotifDate, Type, Status)

FK ApplicantID --> APPLICANT(UserID)

---

27. **COMPLAINT** (**ComplaintID**, *ApplicantID*, Description, ComplaintDate, Status)

FK ApplicantID --> APPLICANT(UserID)

---

28. **REPORT** (**ReportID**, *AdminID*, GenDate, Type, Content)

FK AdminID --> SYSTEM_ADMIN(UserID)

A.4 Schema Refinement Notes

- Reserved - keyword avoidance: USER --> USERS; DATE/TIME columns renamed (e.g. SlotDate, AppointmentTime).
- ENUM replacement: SQL Server has no ENUM type, so Status/Type/Role/BloodGroup use VARCHAR + CHECK constraints.
- All 28 tables are in 3NF - every non-key attribute depends only on the whole primary key.
- SupervisorID (self-referencing FK) uses ON DELETE NO ACTION - SQL Server disallows cascading self-references.

Part B - SQL DDL Implementation

All 28 tables from the schema above, implemented in SQL Server (T-SQL) with primary keys, foreign keys, CHECK constraints and cascade rules.

```
CREATE DATABASE DriveLink;
USE DriveLink;
```

```
-- TABLE 1: USERS (renamed from USER, a reserved word). Parent of the ISA hierarchy.
CREATE TABLE USERS (
    UserID     INT           IDENTITY(1,1) PRIMARY KEY,
    FirstName  VARCHAR(50)   NOT NULL,
    LastName   VARCHAR(50)   NOT NULL,
    Email      VARCHAR(100)  NOT NULL UNIQUE,
    Password   VARCHAR(255)  NOT NULL,
    Role       VARCHAR(15)   NOT NULL
               CHECK (Role IN
('Applicant','Officer','Coordinator','Examiner','Trainer','MedOfficer','Admin')),
    Street     VARCHAR(100),
    City       VARCHAR(50),
    Province   VARCHAR(50),
    PostalCode VARCHAR(10)
);
```

```
-- TABLE 2: USER_PHONE. Multivalued attribute {Phone} of USER.
CREATE TABLE USER_PHONE (
    UserID INT         NOT NULL,
    Phone  VARCHAR(15) NOT NULL,
    PRIMARY KEY (UserID, Phone),
    FOREIGN KEY (UserID) REFERENCES USERS(UserID) ON DELETE CASCADE ON UPDATE CASCADE
);
```

```
-- TABLE 3: APPLICANT. ISA subtype of USER (disjoint).CREATE TABLE APPLICANT (
    UserID     INT         PRIMARY KEY,
    NIC        VARCHAR(12) NOT NULL UNIQUE,
    DOB        DATE        NOT NULL,
    BloodGroup VARCHAR(5)  NOT NULL CHECK (BloodGroup IN ('A+','A-','B+','B-','AB+','AB-
','O+','O-')),
    FOREIGN KEY (UserID) REFERENCES USERS(UserID) ON DELETE CASCADE ON UPDATE CASCADE
);
```

```
-- TABLE 4: EXISTING_LICENSE_HOLDER. ISA subtype of APPLICANT (disjoint).
CREATE TABLE EXISTING_LICENSE_HOLDER (
    UserID     INT         PRIMARY KEY,
    LicenseNo  VARCHAR(20) NOT NULL UNIQUE,
    ExpiryDate DATE        NOT NULL,
    FOREIGN KEY (UserID) REFERENCES APPLICANT(UserID) ON DELETE CASCADE ON UPDATE
CASCADE
);
```

```
-- TABLE 5: REGISTRATION_OFFICER. ISA subtype of USER (disjoint).
CREATE TABLE REGISTRATION_OFFICER (
    UserID     INT         PRIMARY KEY,
    OfficerID  VARCHAR(15) NOT NULL UNIQUE,
    Department VARCHAR(100),
    FOREIGN KEY (UserID) REFERENCES USERS(UserID) ON DELETE CASCADE ON UPDATE CASCADE
);
```

```
-- TABLE 6: TEST_COORDINATOR. ISA subtype of USER (disjoint).
CREATE TABLE TEST_COORDINATOR (
    UserID  INT         PRIMARY KEY,
    CoordID VARCHAR(15) NOT NULL UNIQUE,
    FOREIGN KEY (UserID) REFERENCES USERS(UserID) ON DELETE CASCADE ON UPDATE CASCADE
);
```

```
-- TABLE 7: DRIVING_EXAMINER. ISA subtype of USER; SupervisorID = recursive "supervises"
(1:N).
CREATE TABLE DRIVING_EXAMINER (
    UserID        INT         PRIMARY KEY,
    ExaminerID    VARCHAR(15) NOT NULL UNIQUE,
    Qualification VARCHAR(100),
    SupervisorID  INT         NULL,
    FOREIGN KEY (UserID) REFERENCES USERS(UserID) ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (SupervisorID) REFERENCES DRIVING_EXAMINER(UserID) ON DELETE NO ACTION
ON UPDATE NO ACTION
);
```

```
-- TABLE 8: DRIVING_TRAINER. ISA subtype of USER (disjoint).
CREATE TABLE DRIVING_TRAINER (
    UserID         INT         PRIMARY KEY,
    TrainerID      VARCHAR(15) NOT NULL UNIQUE,
    Specialization VARCHAR(100),
    FOREIGN KEY (UserID) REFERENCES USERS(UserID) ON DELETE CASCADE ON UPDATE CASCADE
);
```

```
-- TABLE 9: MEDICAL_OFFICER. ISA subtype of USER (disjoint).
CREATE TABLE MEDICAL_OFFICER (
    UserID       INT         PRIMARY KEY,
    MedOfficerID VARCHAR(15) NOT NULL UNIQUE,
    Designation  VARCHAR(100),
    FOREIGN KEY (UserID) REFERENCES USERS(UserID) ON DELETE CASCADE ON UPDATE CASCADE
);
```

```
-- TABLE 10: SYSTEM_ADMIN. ISA subtype of USER (disjoint).
CREATE TABLE SYSTEM_ADMIN (
    UserID  INT         PRIMARY KEY,
    AdminID VARCHAR(15) NOT NULL UNIQUE,
    FOREIGN KEY (UserID) REFERENCES USERS(UserID) ON DELETE CASCADE ON UPDATE CASCADE
);
```

```
 -- TABLE 11: LICENSE_APPLICATION. Applicant submits; Officer reviews (OfficerID
nullable).
CREATE TABLE LICENSE_APPLICATION (
    ApplicationID   INT IDENTITY(1,1) PRIMARY KEY,
    ApplicantID     INT         NOT NULL,
    OfficerID       INT         NULL,
    SubmitDate      DATE        NOT NULL,
    Status          VARCHAR(15) NOT NULL DEFAULT 'Pending'
                    CHECK (Status IN ('Pending','Under
Review','Approved','Rejected','Issued')),
    LicenseCate ry VARCHAR(5)  NOT NULL CHECK (LicenseCate ry IN
('A1','A','B','C','CE','D')),
    FOREIGN KEY (ApplicantID) REFERENCES APPLICANT(UserID),
    FOREIGN KEY (OfficerID) REFERENCES REGISTRATION_OFFICER(UserID)
);
```

```
-- TABLE 12: DOCUMENT. Application has N documents.
CREATE TABLE DOCUMENT (
    DocumentID    INT IDENTITY(1,1) PRIMARY KEY,
    ApplicationID INT          NOT NULL,
    Type          VARCHAR(20)  NOT NULL CHECK (Type IN
('NIC','Photo','MedicalReport','BirthCertificate','Other')),
    FilePath      VARCHAR(255) NOT NULL,
    UploadDate    DATE         NOT NULL,
    Status        VARCHAR(10)  NOT NULL DEFAULT 'Pending' CHECK (Status IN
('Pending','Verified','Rejected')),
    FOREIGN KEY (ApplicationID) REFERENCES LICENSE_APPLICATION(ApplicationID) ON DELETE
CASCADE ON UPDATE CASCADE
);
```

```
-- TABLE 13: MEDICAL_APPOINTMENT. Applicant books; MedicalOfficer manages.
CREATE TABLE MEDICAL_APPOINTMENT (
    AppointmentID   INT IDENTITY(1,1) PRIMARY KEY,
    ApplicantID     INT         NOT NULL,
    MedOfficerID    INT         NULL,
    AppointmentDate DATE        NOT NULL,
    AppointmentTime TIME        NOT NULL,
    Status          VARCHAR(10) NOT NULL DEFAULT 'Booked' CHECK (Status IN
('Booked','Completed','Cancelled')),
    FOREIGN KEY (ApplicantID) REFERENCES APPLICANT(UserID),
    FOREIGN KEY (MedOfficerID) REFERENCES MEDICAL_OFFICER(UserID)
);
```

```
-- TABLE 14: MEDICAL_TEST. UNIQUE(AppointmentID) enforces the 1:1 with
MEDICAL_APPOINTMENT.
CREATE TABLE MEDICAL_TEST (
    MedTestID     INT IDENTITY(1,1) PRIMARY KEY,
    AppointmentID INT         NOT NULL UNIQUE,
    TestDate      DATE        NOT NULL,
```

```
    Result        VARCHAR(10) NOT NULL DEFAULT 'Pending' CHECK (Result IN
('Pass','Fail','Pending')),
    Status        VARCHAR(10) NOT NULL DEFAULT 'Scheduled' CHECK (Status IN
('Scheduled','Completed','Cancelled')),
    FOREIGN KEY (AppointmentID) REFERENCES MEDICAL_APPOINTMENT(AppointmentID) ON DELETE
CASCADE ON UPDATE CASCADE
);
```

```
-- TABLE 15: EXAM_SLOT. TestCoordinator creates N slots.
CREATE TABLE EXAM_SLOT (
    SlotID   INT IDENTITY(1,1) PRIMARY KEY,
    CoordID  INT          NOT NULL,
    SlotDate DATE         NOT NULL,
    SlotTime TIME         NOT NULL,
    Location VARCHAR(100) NOT NULL,
    Capacity INT          NOT NULL CHECK (Capacity > 0),
    FOREIGN KEY (CoordID) REFERENCES TEST_COORDINATOR(UserID)
);
```

```
-- TABLE 16: DRIVING_EXAM. Applicant books; DrivingExaminer conducts.
CREATE TABLE DRIVING_EXAM (
    ExamID      INT IDENTITY(1,1) PRIMARY KEY,
    ApplicantID INT          NOT NULL,
    ExaminerID  INT          NULL,
    ExamDate    DATE         NOT NULL,
    Location    VARCHAR(100) NOT NULL,
    Type        VARCHAR(10)  NOT NULL DEFAULT 'Theory' CHECK (Type IN
('Theory','Written')),
    Score       DECIMAL(5,2) NULL,
    Status      VARCHAR(10)  NOT NULL DEFAULT 'Booked' CHECK (Status IN
('Booked','Passed','Failed','Cancelled')),
    FOREIGN KEY (ApplicantID) REFERENCES APPLICANT(UserID),
    FOREIGN KEY (ExaminerID) REFERENCES DRIVING_EXAMINER(UserID)
);
```

```
-- TABLE 17: EXAM_SLOT_ASSIGNMENT. Junction table for M:N DRIVING_EXAM <-> EXAM_SLOT.
CREATE TABLE EXAM_SLOT_ASSIGNMENT (
    ExamID INT NOT NULL,
    SlotID INT NOT NULL,
    PRIMARY KEY (ExamID, SlotID),
    FOREIGN KEY (ExamID) REFERENCES DRIVING_EXAM(ExamID) ON DELETE CASCADE ON UPDATE
CASCADE,
    FOREIGN KEY (SlotID) REFERENCES EXAM_SLOT(SlotID) ON DELETE CASCADE ON UPDATE
CASCADE
);
```

```
-- TABLE 18: DRIVING_TRIAL. Applicant books; DrivingExaminer conducts.
CREATE TABLE DRIVING_TRIAL (
    TrialID     INT IDENTITY(1,1) PRIMARY KEY,
```

```
    ApplicantID INT          NOT NULL,
    ExaminerID  INT          NULL,
    TrialDate   DATE         NOT NULL,
    Result      VARCHAR(10)  NOT NULL DEFAULT 'Pending' CHECK (Result IN
('Pass','Fail','Pending')),
    Remarks     NVARCHAR(MAX),
    Status      VARCHAR(10)  NOT NULL DEFAULT 'Booked' CHECK (Status IN
('Booked','Completed','Cancelled')),
    FOREIGN KEY (ApplicantID) REFERENCES APPLICANT(UserID),
    FOREIGN KEY (ExaminerID) REFERENCES DRIVING_EXAMINER(UserID)
);
```

```
-- TABLE 19: TRAINING_SESSION. DrivingTrainer conducts N sessions.
CREATE TABLE TRAINING_SESSION (
    SessionID   INT IDENTITY(1,1) PRIMARY KEY,
    TrainerID   INT          NOT NULL,
    SessionDate DATE         NOT NULL,
    Progress    VARCHAR(100),
    Remarks     NVARCHAR(MAX),
    FOREIGN KEY (TrainerID) REFERENCES DRIVING_TRAINER(UserID)
);
```

```
-- TABLE 20: TRAINER_APPLICANT. Junction table for M:N; AssignDate is the relationship
attribute.
CREATE TABLE TRAINER_APPLICANT (
    TrainerID   INT  NOT NULL,
    ApplicantID INT  NOT NULL,
    AssignDate  DATE NOT NULL,
    PRIMARY KEY (TrainerID, ApplicantID),
    FOREIGN KEY (TrainerID) REFERENCES DRIVING_TRAINER(UserID) ON DELETE CASCADE ON
UPDATE CASCADE,
    FOREIGN KEY (ApplicantID) REFERENCES APPLICANT(UserID)
);
```

```
-- TABLE 21: APPLICANT_SESSION. Junction table for M:N APPLICANT <-> TRAINING_SESSION.
CREATE TABLE APPLICANT_SESSION (
    ApplicantID INT NOT NULL,
    SessionID   INT NOT NULL,
    PRIMARY KEY (ApplicantID, SessionID),
    FOREIGN KEY (ApplicantID) REFERENCES APPLICANT(UserID) ON DELETE CASCADE ON UPDATE
CASCADE,
    FOREIGN KEY (SessionID) REFERENCES TRAINING_SESSION(SessionID) ON DELETE CASCADE ON
UPDATE CASCADE
);
```

```
-- TABLE 22: DRIVING_LICENSE. UNIQUE(ApplicantID) enforces the 1:1 "receives"; Officer
issues.
CREATE TABLE DRIVING_LICENSE (
    LicenseID   INT IDENTITY(1,1) PRIMARY KEY,
```

```
    ApplicantID INT         NOT NULL UNIQUE,
    OfficerID   INT         NULL,
    IssueDate   DATE        NOT NULL,
    ExpiryDate  DATE        NOT NULL,
    Type        VARCHAR(10) NOT NULL DEFAULT 'Normal' CHECK (Type IN
('Normal','OneDay')),
    FOREIGN KEY (ApplicantID) REFERENCES APPLICANT(UserID),
    FOREIGN KEY (OfficerID) REFERENCES REGISTRATION_OFFICER(UserID)
);
```

```
-- TABLE 23: LICENSE_VEHICLE_CATE RY. Multivalued attribute {VehicleCate ries}.
CREATE TABLE LICENSE_VEHICLE_CATE RY (
    LicenseID       INT        NOT NULL,
    VehicleCate ry VARCHAR(5) NOT NULL CHECK (VehicleCate ry IN
('A1','A','B','C','CE','D')),
    PRIMARY KEY (LicenseID, VehicleCate ry),
    FOREIGN KEY (LicenseID) REFERENCES DRIVING_LICENSE(LicenseID) ON DELETE CASCADE ON
UPDATE CASCADE
);
```

```
-- TABLE 24: RENEWAL_REQUEST. LicenseHolder submits; Officer processes.
CREATE TABLE RENEWAL_REQUEST (
    RenewalID       INT IDENTITY(1,1) PRIMARY KEY,
    LicenseHolderID INT         NOT NULL,
    OfficerID       INT         NULL,
    RequestDate     DATE        NOT NULL,
    Status          VARCHAR(10) NOT NULL DEFAULT 'Pending' CHECK (Status IN
('Pending','Approved','Rejected')),
    FOREIGN KEY (LicenseHolderID) REFERENCES EXISTING_LICENSE_HOLDER(UserID),
    FOREIGN KEY (OfficerID) REFERENCES REGISTRATION_OFFICER(UserID)
);
```

```
-- TABLE 25: PAYMENT. Applicant makes N payments.
CREATE TABLE PAYMENT (
    PaymentID   INT IDENTITY(1,1) PRIMARY KEY,
    ApplicantID INT           NOT NULL,
    Amount      DECIMAL(10,2) NOT NULL CHECK (Amount > 0),
    PaymentDate DATE          NOT NULL,
    Method      VARCHAR(15)   NOT NULL CHECK (Method IN ('Card','BankTransfer','Cash')),
    Status      VARCHAR(10)   NOT NULL DEFAULT 'Pending' CHECK (Status IN
('Paid','Pending','Failed')),
    Type        VARCHAR(15)   NOT NULL CHECK (Type IN
('Application','MedicalTest','DrivingExam','DrivingTrial','Renewal','OneDay')),
    FOREIGN KEY (ApplicantID) REFERENCES APPLICANT(UserID)
);
```

```
-- TABLE 26: NOTIFICATION. Applicant receives N notifications.
CREATE TABLE NOTIFICATION (
    NotifID     INT IDENTITY(1,1) PRIMARY KEY,
```

```
    ApplicantID INT           NOT NULL,
    Message     NVARCHAR(MAX) NOT NULL,
    NotifDate   DATETIME      NOT NULL DEFAULT GETDATE(),
    Type        VARCHAR(15)   NOT NULL CHECK (Type IN
('Booking','Approval','Rejection','Reminder','ExpiryAlert')),
    Status      VARCHAR(10)   NOT NULL DEFAULT 'Unread' CHECK (Status IN
('Unread','Read')),
    FOREIGN KEY (ApplicantID) REFERENCES APPLICANT(UserID)
);
```

```
-- TABLE 27: COMPLAINT. Applicant submits N complaints.
CREATE TABLE COMPLAINT (
    ComplaintID   INT IDENTITY(1,1) PRIMARY KEY,
    ApplicantID   INT           NOT NULL,
    Description   NVARCHAR(MAX) NOT NULL,
    ComplaintDate DATE          NOT NULL,
    Status        VARCHAR(15)   NOT NULL DEFAULT 'Open' CHECK (Status IN
('Open','InProgress','Resolved')),
    FOREIGN KEY (ApplicantID) REFERENCES APPLICANT(UserID)
);
```

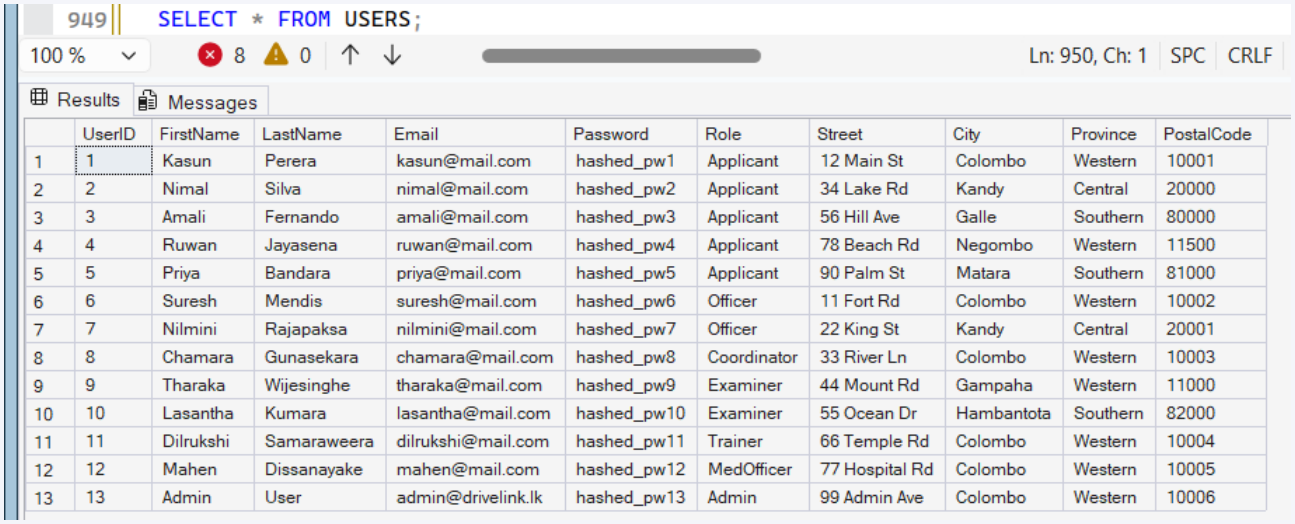
```
-- TABLE 28: REPORT. SystemAdmin generates N reports.
CREATE TABLE REPORT (
    ReportID INT IDENTITY(1,1) PRIMARY KEY,
    AdminID  INT          NOT NULL,
    GenDate  DATE         NOT NULL,
    Type     VARCHAR(15)  NOT NULL CHECK (Type IN
('Application','Payment','Exam','License','Audit')),
    Content  NVARCHAR(MAX),
    FOREIGN KEY (AdminID) REFERENCES SYSTEM_ADMIN(UserID)
);
```

Part C - Insert Sample Data

Each table is populated with at least 5 valid records respecting all CHECK/FK constraints.

```
USE DriveLink;

-- USERS
INSERT INTO USERS (FirstName, LastName, Email, Password, Role, Street, City, Province,
PostalCode) VALUES
('Kasun','Perera','kasun@mail.com','hashed_pw1','Applicant','12 Main
St','Colombo','Western','10001'),
('Nimal','Silva','nimal@mail.com','hashed_pw2','Applicant','34 Lake
Rd','Kandy','Central','20000'),
('Amali','Fernando','amali@mail.com','hashed_pw3','Applicant','56 Hill
Ave','Galle','Southern','80000'),
('Ruwan','Jayasena','ruwan@mail.com','hashed_pw4','Applicant','78 Beach Rd','Ne
mbo','Western','11500'),
('Priya','Bandara','priya@mail.com','hashed_pw5','Applicant','90 Palm
St','Matara','Southern','81000'),
('Suresh','Mendis','suresh@mail.com','hashed_pw6','Officer','11 Fort
Rd','Colombo','Western','10002'),
('Nilmini','Rajapaksa','nilmini@mail.com','hashed_pw7','Officer','22 King
St','Kandy','Central','20001'),
('Chamara','Gunasekara','chamara@mail.com','hashed_pw8','Coordinator','33 River
Ln','Colombo','Western','10003'),
('Tharaka','Wijesinghe','tharaka@mail.com','hashed_pw9','Examiner','44 Mount
Rd','Gampaha','Western','11000'),
('Lasantha','Kumara','lasantha@mail.com','hashed_pw10','Examiner','55 Ocean
Dr','Hambantota','Southern','82000'),
('Dilrukshi','Samaraweera','dilrukshi@mail.com','hashed_pw11','Trainer','66 Temple
Rd','Colombo','Western','10004'),
('Mahen','Dissanayake','mahen@mail.com','hashed_pw12','MedOfficer','77 Hospital
Rd','Colombo','Western','10005'),
 ('Admin','User','admin@drivelink.lk','hashed_pw13','Admin','99 Admin
Ave','Colombo','Western','10006');
```



```
949    SELECT * FROM USERS;
```

100 % | 8 | 0 | Ln: 950, Ch: 1 | SPC | CRLF

Results | Messages

	UserID	FirstName	LastName	Email	Password	Role	Street	City	Province	PostalCode
1	1	Kasun	Perera	kasun@mail.com	hashed_pw1	Applicant	12 Main St	Colombo	Western	10001
2	2	Nimal	Silva	nimal@mail.com	hashed_pw2	Applicant	34 Lake Rd	Kandy	Central	20000
3	3	Amali	Fernando	amali@mail.com	hashed_pw3	Applicant	56 Hill Ave	Galle	Southern	80000
4	4	Ruwan	Jayasena	ruwan@mail.com	hashed_pw4	Applicant	78 Beach Rd	Negombo	Western	11500
5	5	Priya	Bandara	priya@mail.com	hashed_pw5	Applicant	90 Palm St	Matara	Southern	81000
6	6	Suresh	Mendis	suresh@mail.com	hashed_pw6	Officer	11 Fort Rd	Colombo	Western	10002
7	7	Nilmini	Rajapaksa	nilmini@mail.com	hashed_pw7	Officer	22 King St	Kandy	Central	20001
8	8	Chamara	Gunasekara	chamara@mail.com	hashed_pw8	Coordinator	33 River Ln	Colombo	Western	10003
9	9	Tharaka	Wijesinghe	tharaka@mail.com	hashed_pw9	Examiner	44 Mount Rd	Gampaha	Western	11000
10	10	Lasantha	Kumara	lasantha@mail.com	hashed_pw10	Examiner	55 Ocean Dr	Hambantota	Southern	82000
11	11	Dilrukshi	Samaraweera	dilrukshi@mail.com	hashed_pw11	Trainer	66 Temple Rd	Colombo	Western	10004
12	12	Mahen	Dissanayake	mahen@mail.com	hashed_pw12	MedOfficer	77 Hospital Rd	Colombo	Western	10005
13	13	Admin	User	admin@drivelink.lk	hashed_pw13	Admin	99 Admin Ave	Colombo	Western	10006

```
-- USER_PHONE
INSERT INTO USER_PHONE (UserID, Phone) VALUES
(1,'0771234567'),(1,'0112345678'),(2,'0762345678'),(3,'0753456789'),(4,'0744567890'),
(5,'0735678901'),(6,'0726789012'),(7,'0717890123'),(8,'0708901234'),(12,'0119012345');
```

```
933  SELECT * FROM USER_PHONE;
```

110 % 21 0

Results Messages

	UserID	Phone
1	1	0112345678
2	1	0771234567
3	2	0762345678
4	3	0753456789
5	4	0744567890
6	5	0735678901
7	6	0726789012
8	7	0717890123
9	8	0708901234
10	12	0119012345

```
-- APPLICANT
INSERT INTO APPLICANT (UserID, NIC, DOB, BloodGroup) VALUES
(1,'200012345678','2000-03-15','B+'),(2,'199823456789','1998-07-22','A+'),
(3,'200134567890','2001-11-05','O-'),(4,'199945678901','1999-01-30','AB+'),
(5,'200256789012','2002-06-18','B-');
```

```
955  SELECT * FROM APPLICANT;
```

100 % 8 0

Results Messages

	UserID	NIC	DOB	BloodGroup
1	1	200012345678	2000-03-15	B+
2	2	199823456789	1998-07-22	A+
3	3	200134567890	2001-11-05	O-
4	4	199945678901	1999-01-30	AB+
5	5	200256789012	2002-06-18	B-

```
-- EXISTING_LICENSE_HOLDER
INSERT INTO EXISTING_LICENSE_HOLDER (UserID, LicenseNo, ExpiryDate) VALUES
(4,'LIC-2019-00456','2025-01-30'),(5,'LIC-2020-00789','2026-06-18');
```

```
958  SELECT * FROM EXISTING_LICENSE_HOLDER;
```

100 % 8 0

Results Messages

	UserID	LicenseNo	ExpiryDate
1	4	LIC-2019-00456	2025-01-30
2	5	LIC-2020-00789	2026-06-18

```
-- REGISTRATION_OFFICER
INSERT INTO REGISTRATION_OFFICER (UserID, OfficerID, Department) VALUES
(6,'OFF-001','License Registration'),(7,'OFF-002','Document Verification');
```

```
961  SELECT * FROM REGISTRATION_OFFICER;
```

100 % 8 0

Results Messages

	UserID	OfficerID	Department
1	6	OFF-001	License Registration
2	7	OFF-002	Document Verification

```
-- TEST_COORDINATOR
INSERT INTO TEST_COORDINATOR (UserID, CoordID) VALUES (8,'COORD-001');
```

```
964  SELECT * FROM TEST_COORDINATOR;
```

100 % 8 0

Results Messages

	UserID	CoordID
1	8	COORD-001

```
-- DRIVING_EXAMINER (UserID 10 supervised by UserID 9 -- recursive relationship)
INSERT INTO DRIVING_EXAMINER (UserID, ExaminerID, Qualification, SupervisorID) VALUES
(9,'EXAM-001','Senior Certified Examiner',NULL),(10,'EXAM-002','Certified Examiner',9);
```

```
948  SELECT * FROM DRIVING_EXAMINER;
```

110 % 21 0

Results Messages

	UserID	ExaminerID	Qualification	SupervisorID
1	9	EXAM-001	Senior Certified Examiner	NULL
2	10	EXAM-002	Certified Examiner	9

```
-- DRIVING_TRAINER
INSERT INTO DRIVING_TRAINER VALUES (11,'TRN-001','Motorcycle and Light Vehicle');
```

```
951  SELECT * FROM DRIVING_TRAINER;
```

110 % 21 0

Results Messages

	UserID	TrainerID	Specialization
1	11	TRN-001	Motorcycle and Light Vehicle

```
-- MEDICAL_OFFICER
INSERT INTO MEDICAL_OFFICER VALUES (12,'MED-001',' vernment Medical Officer');
```

```
954  SELECT * FROM MEDICAL_OFFICER;
```

110 % 21 0

Results Messages

	UserID	MedOfficerID	Designation
1	12	MED-001	Government Medical Officer

```
-- SYSTEM_ADMIN
INSERT INTO SYSTEM_ADMIN VALUES (13,'ADM-001');
```

```
957    SELECT * FROM SYSTEM_ADMIN;
```

110 % 21 0

Results Messages

	UserID	AdminID
1	13	ADM-001

```
-- LICENSE_APPLICATION
INSERT INTO LICENSE_APPLICATION (ApplicantID, OfficerID, SubmitDate, Status, LicenseCate
ry) VALUES
(1,6,'2026-01-10','Approved','B'),(2,6,'2026-01-15','Under Review','A1'),
(3,7,'2026-02-01','Pending','B'),(4,7,'2026-02-10','Approved','C'),
(5,6,'2026-02-20','Rejected','B');
```

```
979    SELECT * FROM LICENSE_APPLICATION;
```

100 % 8 0

Results Messages

	ApplicationID	ApplicantID	OfficerID	SubmitDate	Status	LicenseCategory
1	1	1	6	2026-01-10	Approved	B
2	2	2	6	2026-01-15	Under Review	A1
3	3	3	7	2026-02-01	Pending	B
4	4	4	7	2026-02-10	Approved	C
5	5	5	6	2026-02-20	Rejected	B

```
-- DOCUMENT
INSERT INTO DOCUMENT (ApplicationID, Type, FilePath, UploadDate, Status) VALUES
(1,'NIC','/docs/app1/nic.pdf','2026-01-
10','Verified'),(1,'Photo','/docs/app1/photo.jpg','2026-01-10','Verified'),
(1,'MedicalReport','/docs/app1/med.pdf','2026-01-
10','Verified'),(2,'NIC','/docs/app2/nic.pdf','2026-01-15','Pending'),
(2,'Photo','/docs/app2/photo.jpg','2026-01-
15','Verified'),(3,'NIC','/docs/app3/nic.pdf','2026-02-01','Pending'),
(3,'MedicalReport','/docs/app3/med.pdf','2026-02-
01','Pending'),(4,'NIC','/docs/app4/nic.pdf','2026-02-10','Verified'),
(4,'MedicalReport','/docs/app4/med.pdf','2026-02-
10','Verified'),(5,'NIC','/docs/app5/nic.pdf','2026-02-20','Rejected');
```

```
982    SELECT * FROM DOCUMENT;
```

100 % 8 0

Results Messages

	DocumentID	ApplicationID	Type	FilePath	UploadDate	Status
1	1	1	NIC	/docs/app1/nic.pdf	2026-01-10	Verified
2	2	1	Photo	/docs/app1/photo.jpg	2026-01-10	Verified
3	3	1	MedicalReport	/docs/app1/med.pdf	2026-01-10	Verified
4	4	2	NIC	/docs/app2/nic.pdf	2026-01-15	Pending
5	5	2	Photo	/docs/app2/photo.jpg	2026-01-15	Verified
6	6	3	NIC	/docs/app3/nic.pdf	2026-02-01	Pending
7	7	3	MedicalReport	/docs/app3/med.pdf	2026-02-01	Pending
8	8	4	NIC	/docs/app4/nic.pdf	2026-02-10	Verified
9	9	4	MedicalReport	/docs/app4/med.pdf	2026-02-10	Verified
10	10	5	NIC	/docs/app5/nic.pdf	2026-02-20	Rejected

```
-- MEDICAL_APPOINTMENT
INSERT INTO MEDICAL_APPOINTMENT (ApplicantID, MedOfficerID, AppointmentDate,
AppointmentTime, Status) VALUES
(1,12,'2025-12-15','09:00:00','Completed'),(2,12,'2026-01-05','10:00:00','Completed'),
(3,12,'2026-01-20','11:00:00','Booked'),(4,12,'2026-02-05','09:30:00','Completed'),
(5,12,'2026-02-18','14:00:00','Cancelled');
```

```
966    SELECT * FROM MEDICAL_APPOINTMENT;
```

110 % 21 0

Results Messages

	AppointmentID	ApplicantID	MedOfficerID	AppointmentDate	AppointmentTime	Status
1	1	1	12	2025-12-15	09:00:00.0000000	Completed
2	2	2	12	2026-01-05	10:00:00.0000000	Completed
3	3	3	12	2026-01-20	11:00:00.0000000	Booked
4	4	4	12	2026-02-05	09:30:00.0000000	Completed
5	5	5	12	2026-02-18	14:00:00.0000000	Cancelled

```
-- MEDICAL_TEST
INSERT INTO MEDICAL_TEST (AppointmentID, TestDate, Result, Status) VALUES
(1,'2025-12-15','Pass','Completed'),(2,'2026-01-05','Pass','Completed'),
(3,'2026-01-20','Pending','Scheduled'),(4,'2026-02-05','Pass','Completed'),
(5,'2026-02-18','Pending','Cancelled');
```

```
969    SELECT * FROM MEDICAL_TEST;
```

110 % 21 0

Results Messages

	MedTestID	AppointmentID	TestDate	Result	Status
1	1	1	2025-12-15	Pass	Completed
2	2	2	2026-01-05	Pass	Completed
3	3	3	2026-01-20	Pending	Scheduled
4	4	4	2026-02-05	Pass	Completed
5	5	5	2026-02-18	Pending	Cancelled

```
-- EXAM_SLOT
INSERT INTO EXAM_SLOT (CoordID, SlotDate, SlotTime, Location, Capacity) VALUES
(8,'2026-02-01','08:00:00','DMT Colombo Hall A',30),(8,'2026-02-01','13:00:00','DMT
Colombo Hall B',25),
(8,'2026-03-01','08:00:00','DMT Kandy Hall 1',20),(8,'2026-03-01','13:00:00','DMT Galle
Hall 1',20),
(8,'2026-04-01','09:00:00','DMT Colombo Hall A',30);
```

```
991    SELECT * FROM EXAM_SLOT;
```

100 % 8 0

Results Messages

	SlotID	CoordID	SlotDate	SlotTime	Location	Capacity
1	1	8	2026-02-01	08:00:00.0000000	DMT Colombo Hall A	30
2	2	8	2026-02-01	13:00:00.0000000	DMT Colombo Hall B	25
3	3	8	2026-03-01	08:00:00.0000000	DMT Kandy Hall 1	20
4	4	8	2026-03-01	13:00:00.0000000	DMT Galle Hall 1	20
5	5	8	2026-04-01	09:00:00.0000000	DMT Colombo Hall A	30

```
-- DRIVING_EXAM
INSERT INTO DRIVING_EXAM (ApplicantID, ExaminerID, ExamDate, Location, Type, Score,
Status) VALUES
(1,9,'2026-02-01','DMT Colombo','Theory',85.00,'Passed'),(2,9,'2026-02-01','DMT
Colombo','Theory',70.00,'Passed'),
(3,10,'2026-03-01','DMT Kandy','Theory',NULL,'Booked'),(4,10,'2026-03-01','DMT
Kandy','Theory',90.00,'Passed'),
(5,9,'2026-02-01','DMT Colombo','Theory',45.00,'Failed');
```

```
994  SELECT * FROM DRIVING_EXAM;
```

100 % ⊗ 8 ⚠ 0

Results | Messages

	ExamID	ApplicantID	ExaminerID	ExamDate	Location	Type	Score	Status
1	1	1	9	2026-02-01	DMT Colombo	Theory	85.00	Passed
2	2	2	9	2026-02-01	DMT Colombo	Theory	70.00	Passed
3	3	3	10	2026-03-01	DMT Kandy	Theory	NULL	Booked
4	4	4	10	2026-03-01	DMT Kandy	Theory	90.00	Passed
5	5	5	9	2026-02-01	DMT Colombo	Theory	45.00	Failed

```
-- EXAM_SLOT_ASSIGNMENT
INSERT INTO EXAM_SLOT_ASSIGNMENT (ExamID, SlotID) VALUES (1,1),(2,1),(3,3),(4,3),(5,2);
```

```
978  SELECT * FROM EXAM_SLOT_ASSIGNMENT;
```

110 % ⊗ 21 ⚠ 0

Results | Messages

	ExamID	SlotID
1	1	1
2	2	1
3	3	3
4	4	3
5	5	2

```
-- DRIVING_TRIAL
INSERT INTO DRIVING_TRIAL (ApplicantID, ExaminerID, TrialDate, Result, Remarks, Status)
VALUES
(1,9,'2026-03-10','Pass',' od control and road awareness','Completed'),
(2,10,'2026-03-15','Fail','Failed to reverse correctly','Completed'),
(4,9,'2026-03-20','Pass','Excellent performance throughout','Completed'),
(3,10,'2026-04-10','Pending',NULL,'Booked'),
(5,9,'2026-04-15','Pending',NULL,'Booked');
```

```
1000  SELECT * FROM DRIVING_TRIAL;
```

100 % ⊗ 8 ⚠ 0

Results | Messages

	TrialID	ApplicantID	ExaminerID	TrialDate	Result	Remarks	Status
1	1	1	9	2026-03-10	Pass	Good control and road awareness	Completed
2	2	2	10	2026-03-15	Fail	Failed to reverse correctly	Completed
3	3	4	9	2026-03-20	Pass	Excellent performance throughout	Completed
4	4	3	10	2026-04-10	Pending	NULL	Booked
5	5	5	9	2026-04-15	Pending	NULL	Booked

```
-- TRAINING_SESSION
INSERT INTO TRAINING_SESSION (TrainerID, SessionDate, Progress, Remarks) VALUES
(11,'2026-01-20','25%','Basic vehicle controls covered'),(11,'2026-01-27','50%','Road
signs and rules covered'),
(11,'2026-02-03','75%','Highway and city driving practiced'),(11,'2026-02-
10','100%','All modules completed successfully'),
(11,'2026-03-01','25%','New batch - basics introduced');
```

```
1003  SELECT * FROM TRAINING_SESSION;
```

100 % 8 0

Results Messages

	SessionID	TrainerID	SessionDate	Progress	Remarks
1	1	11	2026-01-20	25%	Basic vehicle controls covered
2	2	11	2026-01-27	50%	Road signs and rules covered
3	3	11	2026-02-03	75%	Highway and city driving practiced
4	4	11	2026-02-10	100%	All modules completed successfully
5	5	11	2026-03-01	25%	New batch - basics introduced

```
-- TRAINER_APPLICANT
INSERT INTO TRAINER_APPLICANT (TrainerID, ApplicantID, AssignDate) VALUES
(11,1,'2026-01-18'),(11,2,'2026-01-18'),(11,3,'2026-03-01');
```

```
987  SELECT * FROM TRAINER_APPLICANT;
```

110 % 21 0

Results Messages

	TrainerID	ApplicantID	AssignDate
1	11	1	2026-01-18
2	11	2	2026-01-18
3	11	3	2026-03-01

```
-- APPLICANT_SESSION
INSERT INTO APPLICANT_SESSION (ApplicantID, SessionID) VALUES
(1,1),(1,2),(1,3),(1,4),(2,1),(2,2),(3,5);
```

```
990  SELECT * FROM APPLICANT_SESSION;
```

110 % 21 0

Results Messages

	ApplicantID	SessionID
1	1	1
2	1	2
3	1	3
4	1	4
5	2	1
6	2	2
7	3	5

```
-- DRIVING_LICENSE
INSERT INTO DRIVING_LICENSE (ApplicantID, OfficerID, IssueDate, ExpiryDate, Type) VALUES
(1,6,'2026-04-01','2031-04-01','Normal'),(4,7,'2026-03-25','2031-03-25','Normal');
```

```
1012  SELECT * FROM DRIVING_LICENSE;
```

100 % 8 0

Results Messages

	LicenseID	ApplicantID	OfficerID	IssueDate	ExpiryDate	Type
1	1	1	6	2026-04-01	2031-04-01	Normal
2	2	4	7	2026-03-25	2031-03-25	Normal

```
-- LICENSE_VEHICLE_CATEGORY
INSERT INTO LICENSE_VEHICLE_CATE RY (LicenseID, VehicleCate ry) VALUES
(1,'B'),(2,'B'),(2,'C');
```

```
993    SELECT * FROM DRIVING_LICENSE;
```

110 % 21 0

Results Messages

	LicenseID	ApplicantID	OfficerID	IssueDate	ExpiryDate	Type
1	1	1	6	2026-04-01	2031-04-01	Normal
2	2	4	7	2026-03-25	2031-03-25	Normal

```
-- RENEWAL_REQUEST
INSERT INTO RENEWAL_REQUEST (LicenseHolderID, OfficerID, RequestDate, Status) VALUES
(4,6,'2026-01-15','Approved'),(5,7,'2026-02-01','Pending');
```

```
999    SELECT * FROM RENEWAL_REQUEST;
```

110 % 21 0

Results Messages

	RenewalID	LicenseHolderID	OfficerID	RequestDate	Status
1	1	4	6	2026-01-15	Approved
2	2	5	7	2026-02-01	Pending

```
-- PAYMENT
INSERT INTO PAYMENT (ApplicantID, Amount, PaymentDate, Method, Status, Type) VALUES
(1,1500.00,'2026-01-10','Card','Paid','Application'),(1,500.00,'2025-12-
15','BankTransfer','Paid','MedicalTest'),
(1,1000.00,'2026-02-01','Card','Paid','DrivingExam'),(2,1500.00,'2026-01-
15','Cash','Paid','Application'),
(3,1500.00,'2026-02-01','Card','Pending','Application'),(4,2500.00,'2026-02-
10','Card','Paid','Application'),
(5,1500.00,'2026-02-20','Cash','Paid','Application');
```

```
1021    SELECT * FROM PAYMENT;
```

100 % 8 0

Results Messages

	PaymentID	ApplicantID	Amount	PaymentDate	Method	Status	Type
1	1	1	1500.00	2026-01-10	Card	Paid	Application
2	2	1	500.00	2025-12-15	BankTransfer	Paid	MedicalTest
3	3	1	1000.00	2026-02-01	Card	Paid	DrivingExam
4	4	2	1500.00	2026-01-15	Cash	Paid	Application
5	5	3	1500.00	2026-02-01	Card	Pending	Application
6	6	4	2500.00	2026-02-10	Card	Paid	Application
7	7	5	1500.00	2026-02-20	Cash	Paid	Application

```
-- NOTIFICATION
INSERT INTO NOTIFICATION (ApplicantID, Message, NotifDate, Type, Status) VALUES
(1,'Your medical appointment is confirmed for 2025-12-15.','2025-12-10
10:00:00','Booking','Read'),
(1,'Your license application has been approved.','2026-02-15
09:00:00','Approval','Read'),
(2,'Your driving exam is scheduled for 2026-02-01.','2026-01-25
11:00:00','Booking','Read'),
```

```
(3,'Your application is currently under review.','2026-02-05
08:00:00','Reminder','Unread'),
(5,'Your license application has been rejected.','2026-03-01
09:00:00','Rejection','Unread');
```

```
1005    SELECT * FROM NOTIFICATION;
```

110 % 21 0 Ln: 1004, C

Results Messages

	NotifID	ApplicantID	Message	NotifDate	Type	Status
1	1	1	Your medical appointment is confirmed for 2025-1...	2025-12-10 10:00:00.000	Booking	Read
2	2	1	Your license application has been approved.	2026-02-15 09:00:00.000	Approval	Read
3	3	2	Your driving exam is scheduled for 2026-02-01.	2026-01-25 11:00:00.000	Booking	Read
4	4	3	Your application is currently under review.	2026-02-05 08:00:00.000	Reminder	Unread
5	5	5	Your license application has been rejected.	2026-03-01 09:00:00.000	Rejection	Unread

---

```
-- COMPLAINT
INSERT INTO COMPLAINT (ApplicantID, Description, ComplaintDate, Status) VALUES
(2,'My driving exam result was not updated correctly.','2026-02-10','InProgress'),
(3,'Cannot upload documents - file size error.','2026-02-05','Resolved'),
(5,'Application rejected without a clear reason.','2026-03-02','Open'),
(1,'Payment deducted but booking was not confirmed.','2026-01-11','Resolved'),
(4,'Renewal request status not updating on dashboard.','2026-01-20','InProgress');
```

```
1008    SELECT * FROM COMPLAINT;
```

110 % 21 0

Results Messages

	ComplaintID	ApplicantID	Description	ComplaintDate	Status
1	1	2	My driving exam result was not updated correctly.	2026-02-10	InProgress
2	2	3	Cannot upload documents - file size error.	2026-02-05	Resolved
3	3	5	Application rejected without a clear reason.	2026-03-02	Open
4	4	1	Payment deducted but booking was not confirmed.	2026-01-11	Resolved
5	5	4	Renewal request status not updating on dashboard.	2026-01-20	InProgress

---

```
-- REPORT
INSERT INTO REPORT (AdminID, GenDate, Type, Content) VALUES
(13,'2026-01-31','Application','Monthly application summary January 2026. Total: 28.'),
(13,'2026-01-31','Payment','Total revenue January 2026: LKR 45,000. Pending: LKR
3,000.'),
(13,'2026-02-28','Exam','Theory exam pass rate February 2026: 75%.'),
(13,'2026-02-28','License','Licenses issued February 2026: 12 Normal, 2 OneDay.'),
(13,'2026-03-31','Audit','System access audit Q1 2026. No unauthorized access
detected.');
```

```
1010    -- TABLE 28: REPORT
1011    SELECT * FROM REPORT;
```

110 % 21 0

Results Messages

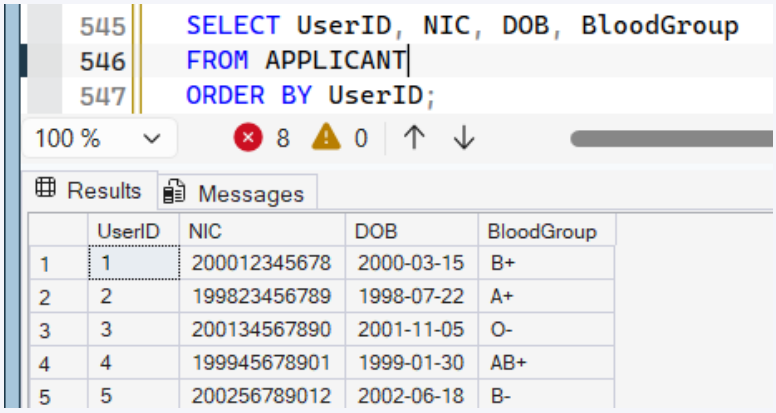
	ReportID	AdminID	GenDate	Type	Content
1	1	13	2026-01-31	Application	Monthly application summary January 2026. Total: 28.
2	2	13	2026-01-31	Payment	Total revenue January 2026: LKR 45,000. Pending: ...
3	3	13	2026-02-28	Exam	Theory exam pass rate February 2026: 75%.
4	4	13	2026-02-28	License	Licenses issued February 2026: 12 Normal, 2 OneD...
5	5	13	2026-03-31	Audit	System access audit Q1 2026. No unauthorized acc...

Part D - SQL Queries and Outputs

Query 1 - Simple SELECT

List all applicants with their personal details (NIC, DOB, blood group) and address, joining the USERS parent table with its APPLICANT subtype.

```
SELECT UserID, NIC, DOB, BloodGroup
FROM APPLICANT
ORDER BY UserID;
```

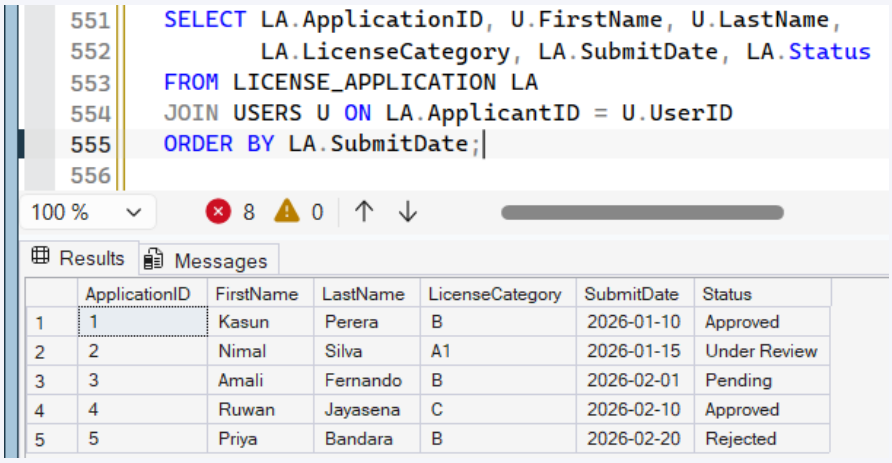


	UserID	NIC	DOB	BloodGroup
1	1	200012345678	2000-03-15	B+
2	2	199823456789	1998-07-22	A+
3	3	200134567890	2001-11-05	O-
4	4	199945678901	1999-01-30	AB+
5	5	200256789012	2002-06-18	B-

Query 2 - JOIN

Produce a license-application report with applicant name, NIC, cate ry, status, and the reviewing officer's name and department.

```
SELECT LA.ApplicationID, U.FirstName, U.LastName,
       LA.LicenseCategory, LA.SubmitDate, LA.Status
FROM LICENSE_APPLICATION LA
JOIN USERS U ON LA.ApplicantID = U.UserID
ORDER BY LA.SubmitDate;
```

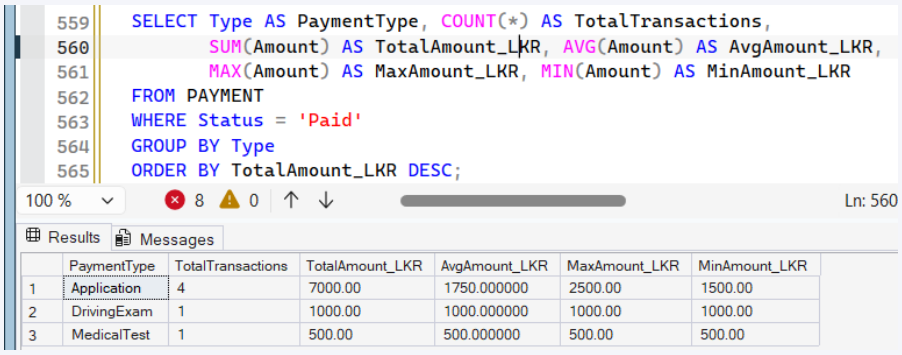


	ApplicationID	FirstName	LastName	LicenseCategory	SubmitDate	Status
1	1	Kasun	Perera	B	2026-01-10	Approved
2	2	Nimal	Silva	A1	2026-01-15	Under Review
3	3	Amali	Fernando	B	2026-02-01	Pending
4	4	Ruwan	Jayasena	C	2026-02-10	Approved
5	5	Priya	Bandara	B	2026-02-20	Rejected

Query 3 - Aggregation

Summarize total transactions, revenue, average, max and min payment per payment type (paid transactions only).

```
SELECT Type AS PaymentType, COUNT(*) AS TotalTransactions,
       SUM(Amount) AS TotalAmount_LKR, AVG(Amount) AS AvgAmount_LKR,
       MAX(Amount) AS MaxAmount_LKR, MIN(Amount) AS MinAmount_LKR
FROM PAYMENT
WHERE Status = 'Paid'
GROUP BY Type
ORDER BY TotalAmount_LKR DESC;
```



```
559  SELECT Type AS PaymentType, COUNT(*) AS TotalTransactions,
560         SUM(Amount) AS TotalAmount_LKR, AVG(Amount) AS AvgAmount_LKR,
561         MAX(Amount) AS MaxAmount_LKR, MIN(Amount) AS MinAmount_LKR
562  FROM PAYMENT
563  WHERE Status = 'Paid'
564  GROUP BY Type
565  ORDER BY TotalAmount_LKR DESC;
```

100 % 8 0 Ln: 560

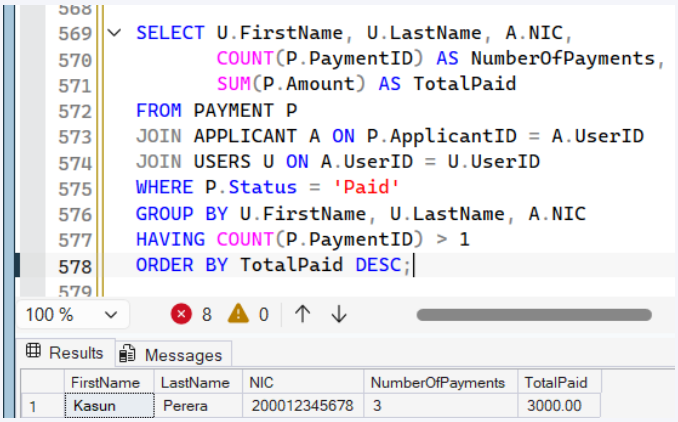
Results Messages

	PaymentType	TotalTransactions	TotalAmount_LKR	AvgAmount_LKR	MaxAmount_LKR	MinAmount_LKR
1	Application	4	7000.00	1750.000000	2500.00	1500.00
2	DrivingExam	1	1000.00	1000.000000	1000.00	1000.00
3	MedicalTest	1	500.00	500.000000	500.00	500.00

Query 4 - GROUP BY / HAVING

Find applicants with more than one successful payment, with their payment count and total paid.

```
SELECT U.FirstName, U.LastName, A.NIC,
       COUNT(P.PaymentID) AS NumberOfPayments,
       SUM(P.Amount) AS TotalPaid
FROM PAYMENT P
JOIN APPLICANT A ON P.ApplicantID = A.UserID
JOIN USERS U ON A.UserID = U.UserID
WHERE P.Status = 'Paid'
GROUP BY U.FirstName, U.LastName, A.NIC
HAVING COUNT(P.PaymentID) > 1
ORDER BY TotalPaid DESC;
```



```
569  SELECT U.FirstName, U.LastName, A.NIC,
570         COUNT(P.PaymentID) AS NumberOfPayments,
571         SUM(P.Amount) AS TotalPaid
572  FROM PAYMENT P
573  JOIN APPLICANT A ON P.ApplicantID = A.UserID
574  JOIN USERS U ON A.UserID = U.UserID
575  WHERE P.Status = 'Paid'
576  GROUP BY U.FirstName, U.LastName, A.NIC
577  HAVING COUNT(P.PaymentID) > 1
578  ORDER BY TotalPaid DESC;
```

100 % 8 0

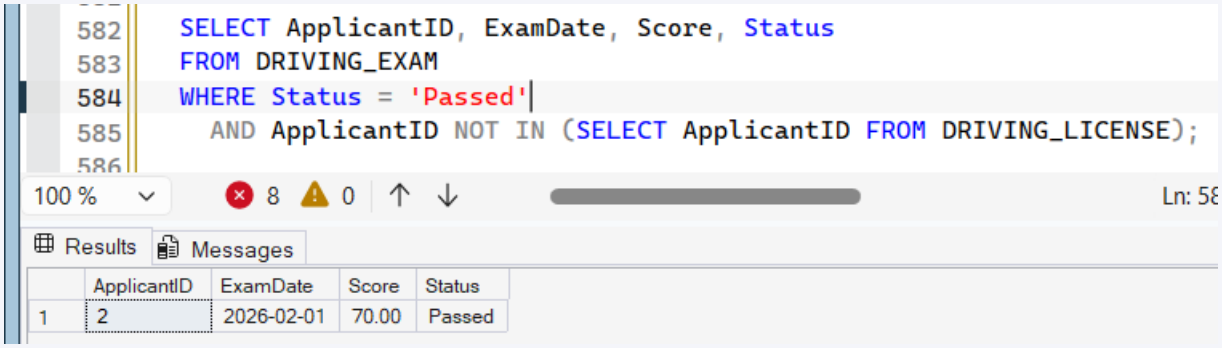
Results Messages

	FirstName	LastName	NIC	NumberOfPayments	TotalPaid
1	Kasun	Perera	200012345678	3	3000.00

Query 5 - Subquery

Identify applicants who passed the driving exam but have not yet received a license.

```
SELECT ApplicantID, ExamDate, Score, Status
FROM DRIVING_EXAM
WHERE Status = 'Passed'
  AND ApplicantID NOT IN (SELECT ApplicantID FROM DRIVING_LICENSE);
```

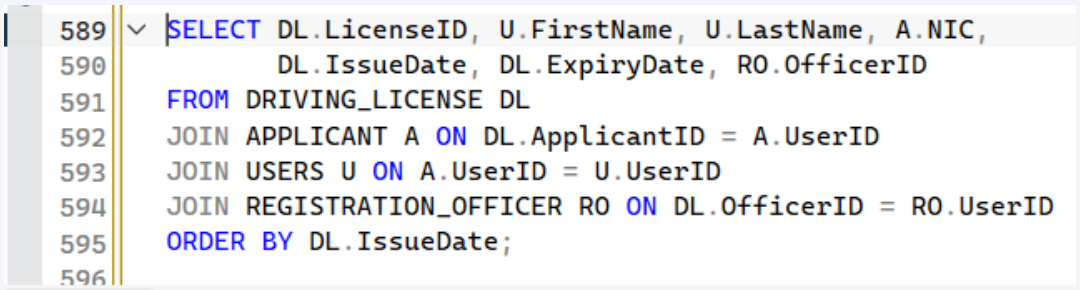


	ApplicantID	ExamDate	Score	Status
1	2	2026-02-01	70.00	Passed

Query 6 - Complex multi-table JOIN

Trace each applicant's full journey: medical result, exam status/score, trial result, and license issuance.

```
SELECT DL.LicenseID, U.FirstName, U.LastName, A.NIC,
       DL.IssueDate, DL.ExpiryDate, RO.OfficerID
FROM DRIVING_LICENSE DL
JOIN APPLICANT A ON DL.ApplicantID = A.UserID
JOIN USERS U ON A.UserID = U.UserID
JOIN REGISTRATION_OFFICER RO ON DL.OfficerID = RO.UserID
ORDER BY DL.IssueDate;
```



	LicenseID	FirstName	LastName	NIC	IssueDate	ExpiryDate	OfficerID
1	2	Ruwan	Jayasena	199945678901	2026-03-25	2031-03-25	OFF-002
2	1	Kasun	Perera	200012345678	2026-04-01	2031-04-01	OFF-001

Part E - Stored Procedures

One procedure per group member, each covering a distinct part of the license-application workflow.

IT25100815 - Simra M.A.F

License Application Status Checker - GetApplicationStatus

Returns the latest application's status for a given applicant NIC.

```
CREATE PROCEDURE GetApplicationStatus
    @NIC VARCHAR(12),
    @Result VARCHAR(300) OUTPUT
AS
BEGIN
    DECLARE @UserID    INT
    DECLARE @AppID     INT
    DECLARE @Status    VARCHAR(20)
    DECLARE @Category  VARCHAR(10)
    DECLARE @Submit    DATE

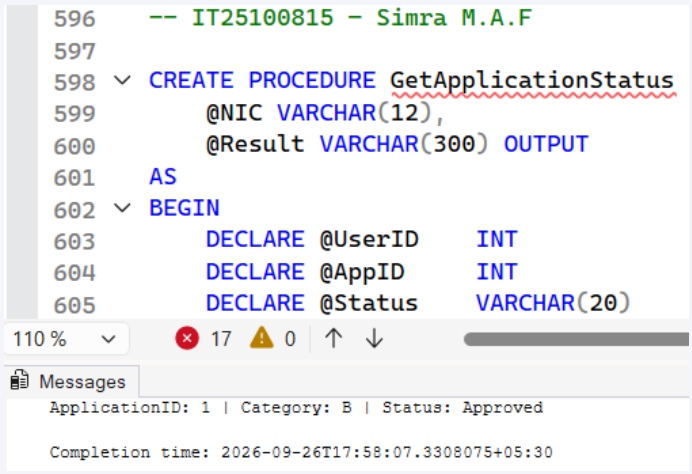
    -- Step 1: Find the applicant
    SELECT @UserID = UserID
    FROM APPLICANT
    WHERE NIC = @NIC

    -- Step 2: Check if applicant exists
    IF @UserID IS NULL
        SET @Result = 'ERROR: No applicant found with this NIC.'
    ELSE
    BEGIN
        -- Step 3: Get their application details
        SELECT TOP 1
               @AppID    = ApplicationID,
               @Status   = Status,
               @Category = LicenseCategory,
               @Submit   = SubmitDate
        FROM LICENSE_APPLICATION
        WHERE ApplicantID = @UserID
        ORDER BY SubmitDate DESC

        -- Step 4: Build result message
        IF @AppID IS NULL
            SET @Result = 'No application found for this applicant.'
        ELSE
            SET @Result = CONCAT('ApplicationID: ', @AppID,
                          ' | Category: ', @Category,
                          ' | Status: ', @Status)
    END
END
GO

-- Calling the procedure
DECLARE @result1 VARCHAR(300)
EXEC GetApplicationStatus @NIC = '200012345678', @Result = @result1 OUTPUT
```

```
PRINT @result1
```



IT25100817 - Weerasinghe W.P.D.V

Medical Appointment Booking - BookMedicalAppointment

Books a medical appointment, blocking duplicate active bookings, and notifies the applicant.

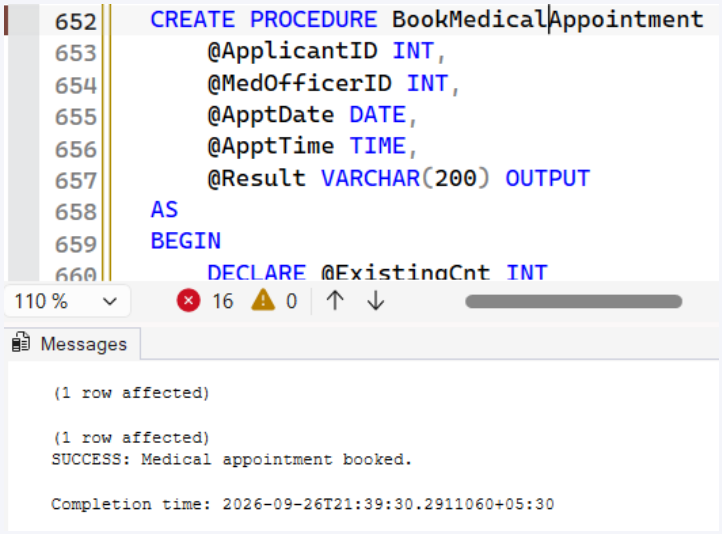
```
CREATE PROCEDURE BookMedicalAppointment
    @ApplicantID INT,
    @MedOfficerID INT,
    @ApptDate DATE,
    @ApptTime TIME,
    @Result VARCHAR(200) OUTPUT
AS
BEGIN
    DECLARE @ExistingCnt INT

    -- Step 1: Check if already has an appointment
    SELECT @ExistingCnt = COUNT(*)
    FROM MEDICAL_APPOINTMENT
    WHERE ApplicantID = @ApplicantID
      AND Status IN ('Booked', 'Completed')

    -- Step 2: Block if already booked
    IF @ExistingCnt > 0
        SET @Result = 'ERROR: Applicant already has a medical appointment.'
    ELSE
    BEGIN
        -- Step 3: Book the appointment
        INSERT INTO MEDICAL_APPOINTMENT
            (ApplicantID, MedOfficerID, AppointmentDate, AppointmentTime, Status)
        VALUES
            (@ApplicantID, @MedOfficerID, @ApptDate, @ApptTime, 'Booked')

        SET @Result = 'SUCCESS: Medical appointment booked.'
    END
END
GO
```

```
-- Calling the procedure
DECLARE @result2 VARCHAR(200)
EXEC BookMedicalAppointment
     @ApplicantID  = 3,
     @MedOfficerID = 12,
     @ApptDate     = '2026-05-01',
     @ApptTime     = '10:00:00',
     @Result       = @result2 OUTPUT
PRINT @result2
```



IT25100821 - Fernando U.A.E.N

Driving Trial Booking - BookDrivingTrial

Books a driving trial only if the applicant has already passed the driving exam.

```
CREATE PROCEDURE BookDrivingTrial
    @ApplicantID INT,
    @ExaminerID  INT,
    @TrialDate   DATE,
    @Result      VARCHAR(200) OUTPUT
AS
BEGIN
    DECLARE @ExamPassed  INT
    DECLARE @TrialExists INT

    -- Step 1: Check if exam is passed
    SELECT @ExamPassed = COUNT(*)
    FROM DRIVING_EXAM
    WHERE ApplicantID = @ApplicantID
      AND Status = 'Passed'

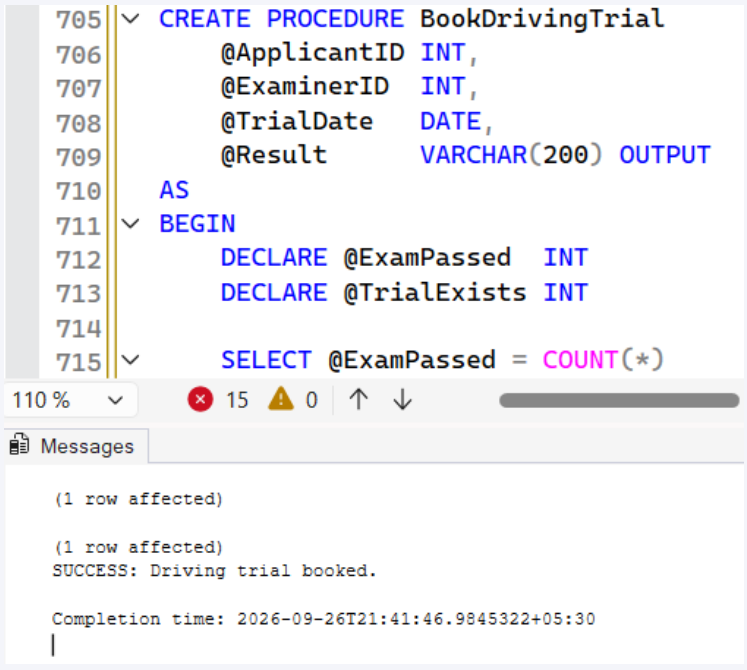
    -- Step 2: Check if trial already booked
    SELECT @TrialExists = COUNT(*)
    FROM DRIVING_TRIAL
```

```
    WHERE ApplicantID = @ApplicantID
      AND Status = 'Booked'

    -- Step 3: Block if exam not passed
    IF @ExamPassed = 0
        SET @Result = 'ERROR: Applicant has not passed the driving exam.'
    -- Step 4: Block if trial already booked
    ELSE IF @TrialExists > 0
        SET @Result = 'ERROR: Applicant already has a driving trial booked.'
    ELSE
    BEGIN
        -- Step 5: Book the trial
        INSERT INTO DRIVING_TRIAL
            (ApplicantID, ExaminerID, TrialDate, Result, Remarks, Status)
        VALUES
            (@ApplicantID, @ExaminerID, @TrialDate, 'Pending', NULL, 'Booked')

        SET @Result = 'SUCCESS: Driving trial booked.'
    END
END
GO

-- Calling the procedure
DECLARE @result3 VARCHAR(200)
EXEC BookDrivingTrial
     @ApplicantID = 2,
     @ExaminerID  = 9,
     @TrialDate   = '2026-05-10',
     @Result      = @result3 OUTPUT
PRINT @result3
```



IT25100830 - Thasalogithan S

License Approval and Issuance - ProcessLicenseApproval

Implements the EER aggregation {DRIVING EXAM + DRIVING TRIAL} ---> LICENSE APPLICATION: checks medical, exam and trial results, then approves and issues the license, or rejects with a reason.

```
CREATE PROCEDURE ProcessLicenseApproval
    @ApplicationID INT,
    @OfficerID     INT,
    @Result        VARCHAR(200) OUTPUT
AS
BEGIN
    DECLARE @ApplicantID INT
    DECLARE @Category    VARCHAR(10)
    DECLARE @AppStatus   VARCHAR(20)
    DECLARE @MedPassed   INT
    DECLARE @ExamPassed  INT
    DECLARE @TrialPassed INT
    DECLARE @LicExists   INT
    DECLARE @NewLicenseID INT

    -- Step 1: Get application details
    SELECT @ApplicantID = ApplicantID,
           @Category    = LicenseCategory,
           @AppStatus   = Status
    FROM LICENSE_APPLICATION
    WHERE ApplicationID = @ApplicationID

    -- Step 2: Check if application exists
    IF @ApplicantID IS NULL
        SET @Result = 'ERROR: Application not found.'

    -- Step 3: Check if already processed
    ELSE IF @AppStatus NOT IN ('Pending', 'Under Review')
        SET @Result = 'ERROR: Application already processed.'

    ELSE
    BEGIN
        -- Step 4: Check medical test
        SELECT @MedPassed = COUNT(*)
        FROM MEDICAL_APPOINTMENT MA
        JOIN MEDICAL_TEST MT ON MA.AppointmentID = MT.AppointmentID
        WHERE MA.ApplicantID = @ApplicantID
          AND MT.Result = 'Pass'

        -- Step 5: Check driving exam
        SELECT @ExamPassed = COUNT(*)
        FROM DRIVING_EXAM
        WHERE ApplicantID = @ApplicantID
          AND Status = 'Passed'

        -- Step 6: Check driving trial
        SELECT @TrialPassed = COUNT(*)
        FROM DRIVING_TRIAL
```

```
        WHERE ApplicantID = @ApplicantID
          AND Result = 'Pass'

        -- Step 7: Check license not already issued
        SELECT @LicExists = COUNT(*)
        FROM DRIVING_LICENSE
        WHERE ApplicantID = @ApplicantID

        IF @LicExists > 0
            SET @Result = 'ERROR: License already issued.'

        ELSE IF @MedPassed = 0
        BEGIN
            UPDATE LICENSE_APPLICATION
            SET Status = 'Rejected'
            WHERE ApplicationID = @ApplicationID

            SET @Result = 'REJECTED: Medical test not passed.'
        END

        ELSE IF @ExamPassed = 0
        BEGIN
            UPDATE LICENSE_APPLICATION
            SET Status = 'Rejected'
            WHERE ApplicationID = @ApplicationID

            SET @Result = 'REJECTED: Driving exam not passed.'
        END

        ELSE IF @TrialPassed = 0
        BEGIN
            UPDATE LICENSE_APPLICATION
            SET Status = 'Rejected'
            WHERE ApplicationID = @ApplicationID

            SET @Result = 'REJECTED: Driving trial not passed.'
        END

        ELSE
        BEGIN
            -- Step 8: Approve and issue license
            UPDATE LICENSE_APPLICATION
            SET Status    = 'Approved',
                OfficerID = @OfficerID
            WHERE ApplicationID = @ApplicationID

            INSERT INTO DRIVING_LICENSE
                (ApplicantID, OfficerID, IssueDate, ExpiryDate, Type)
            VALUES
                (@ApplicantID, @OfficerID,
                 CAST(GETDATE() AS DATE),
                 DATEADD(YEAR, 5, CAST(GETDATE() AS DATE)),
                 'Normal')

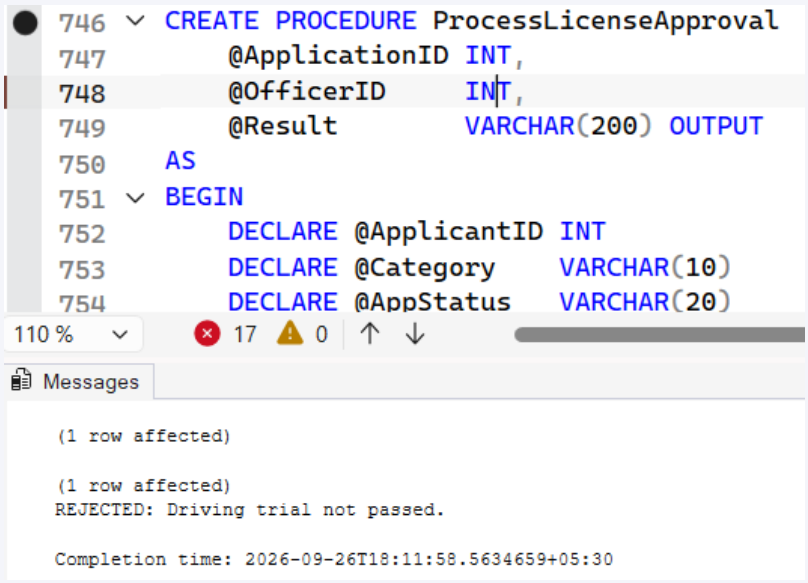
            SET @NewLicenseID = SCOPE_IDENTITY()
```

```
            INSERT INTO LICENSE_VEHICLE_CATEGORY
                (LicenseID, VehicleCategory)
            VALUES
                (@NewLicenseID, @Category)

            SET @Result = 'SUCCESS: License issued for Category ' + @Category
        END
    END
END
GO

-- Calling the procedure
UPDATE LICENSE_APPLICATION SET Status = 'Under Review' WHERE ApplicationID = 2
GO

DECLARE @result4 VARCHAR(200)
EXEC ProcessLicenseApproval
     @ApplicationID = 2,
     @OfficerID     = 6,
     @Result        = @result4 OUTPUT
PRINT @result4
```



IT25100851 - Madampage K.S

Exam Slot Availability Checker - CheckExamSlotAvailability

Returns capacity, current bookings, and remaining seats for a given exam slot.

```
CREATE PROCEDURE CheckExamSlotAvailability
    @SlotID    INT,
    @Result    VARCHAR(200) OUTPUT
AS
BEGIN
    DECLARE @Capacity  INT
    DECLARE @Booked    INT
    DECLARE @Remaining INT
    DECLARE @Location  VARCHAR(100)

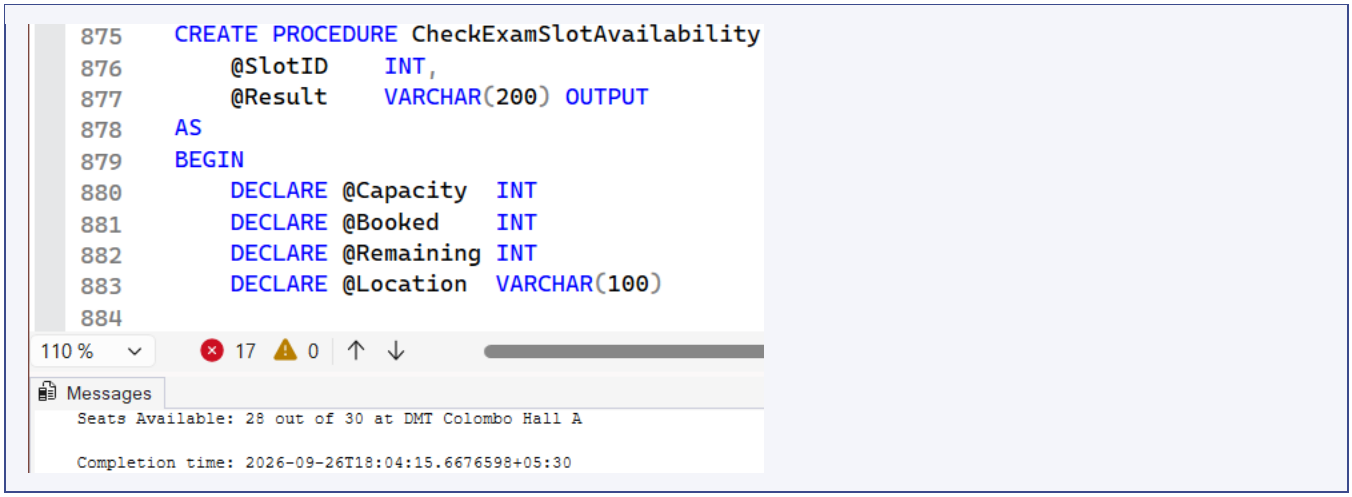
    -- Step 1: Get slot details
    SELECT @Capacity = Capacity,
           @Location = Location
    FROM EXAM_SLOT
    WHERE SlotID = @SlotID

    -- Step 2: Check if slot exists
    IF @Capacity IS NULL
        SET @Result = 'ERROR: Exam slot not found.'
    ELSE
    BEGIN
        -- Step 3: Count how many already booked
        SELECT @Booked = COUNT(*)
        FROM EXAM_SLOT_ASSIGNMENT
        WHERE SlotID = @SlotID

        SET @Remaining = @Capacity - @Booked

        -- Step 4: Return result
        IF @Remaining <= 0
            SET @Result = 'Slot is FULL. No seats available.'
        ELSE
            SET @Result = CONCAT('Seats Available: ', @Remaining,
                          ' out of ', @Capacity,
                          ' at ', @Location)
    END
END
GO

-- Calling the procedure
DECLARE @result5 VARCHAR(200)
EXEC CheckExamSlotAvailability @SlotID = 1, @Result = @result5 OUTPUT
PRINT @result5
```



IT25100856 - Karunarathne D.K.K.P

License Renewal Processing - ProcessLicenseRenewal

Approves a pending renewal, extends license expiry by 5 years, and notifies the holder.

```
CREATE PROCEDURE ProcessLicenseRenewal
    @RenewalID INT,
    @OfficerID INT,
    @Result    VARCHAR(200) OUTPUT
AS
BEGIN
    DECLARE @HolderID   INT
    DECLARE @RenStatus  VARCHAR(20)
    DECLARE @LicenseID  INT
    DECLARE @OldExpiry  DATE
    DECLARE @NewExpiry  DATE

    -- Step 1: Get renewal request
    SELECT @HolderID  = LicenseHolderID,
           @RenStatus = Status
    FROM RENEWAL_REQUEST
    WHERE RenewalID = @RenewalID

    -- Step 2: Check if renewal exists
    IF @HolderID IS NULL
        SET @Result = 'ERROR: Renewal request not found.'

    -- Step 3: Check if still pending
    ELSE IF @RenStatus <> 'Pending'
        SET @Result = 'ERROR: Renewal already processed.'
    ELSE
    BEGIN
        -- Step 4: Get existing license
        SELECT @LicenseID = LicenseID,
               @OldExpiry = ExpiryDate
        FROM DRIVING_LICENSE
        WHERE ApplicantID = @HolderID
```

```
        IF @LicenseID IS NULL
            SET @Result = 'ERROR: No license found for this holder.'
        ELSE
        BEGIN
            -- Step 5: Calculate new expiry
            SET @NewExpiry = DATEADD(YEAR, 5, @OldExpiry)

            -- Step 6: Update all tables
            UPDATE DRIVING_LICENSE
            SET ExpiryDate = @NewExpiry,
                OfficerID  = @OfficerID
            WHERE LicenseID = @LicenseID

            UPDATE RENEWAL_REQUEST
            SET Status    = 'Approved',
                OfficerID = @OfficerID
            WHERE RenewalID = @RenewalID

            UPDATE EXISTING_LICENSE_HOLDER
            SET ExpiryDate = @NewExpiry
            WHERE UserID = @HolderID

            SET @Result = 'SUCCESS: License renewed. New expiry: '
                        + CONVERT(VARCHAR(10), @NewExpiry, 120)
        END
    END
END
GO

-- Calling the procedure
DECLARE @result6 VARCHAR(200)
EXEC ProcessLicenseRenewal
     @RenewalID = 1,
     @OfficerID = 6,
     @Result    = @result6 OUTPUT
PRINT @result6
```

```
938  CREATE PROCEDURE ProcessLicenseRenewal
939      @RenewalID INT,
940      @OfficerID INT,
941      @Result    VARCHAR(200) OUTPUT
942  AS
943  BEGIN
944      DECLARE @HolderID   INT
945      DECLARE @RenStatus  VARCHAR(20)
946      DECLARE @LicenseID  INT
947      DECLARE @OldExpiry  DATE
948      DECLARE @NewExpiry  DATE
949
950      SELECT @HolderID  = LicenseHolderID,
951             @RenStatus = Status
952      FROM RENEWAL_REQUEST
953      WHERE RenewalID = @RenewalID
954
955      IF @HolderID IS NULL
```

110 % 15 0

Messages

```
(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)
SUCCESS: License renewed. New expiry: 2036-03-25

Completion time: 2026-09-26T21:47:03.4666783+05:30
```

Part F - Trigger Implementation

Trigger 1 - trg_auto_notify_exam_result

After a DRIVING_EXAM row is updated to Passed or Failed, automatically inserts a NOTIFICATION for the applicant - removing the need for the application layer to send it manually.

```
CREATE TRIGGER trg_auto_notify_exam_result
ON DRIVING_EXAM
AFTER UPDATE
AS
BEGIN
    DECLARE @ApplicantID INT
    DECLARE @Status      VARCHAR(10)
    DECLARE @ExamDate    DATE
    DECLARE @Score       DECIMAL(5,2)

    -- Step 1: Get the updated row from INSERTED table
    SELECT @ApplicantID = ApplicantID,
           @Status      = Status,
           @ExamDate    = ExamDate,
           @Score       = Score
    FROM INSERTED

    -- Step 2: If status changed to Passed → send pass notification
    IF @Status = 'Passed'
    BEGIN
        INSERT INTO NOTIFICATION
            (ApplicantID, Message, NotifDate, Type, Status)
        VALUES
            (@ApplicantID,
             CONCAT('You PASSED your driving exam on ',
                    CONVERT(VARCHAR(10), @ExamDate, 120),
                    '. Score: ', @Score),
             GETDATE(),
             'Approval',
             'Unread')
    END

    -- Step 3: If status changed to Failed → send fail notification
    IF @Status = 'Failed'
    BEGIN
        INSERT INTO NOTIFICATION
            (ApplicantID, Message, NotifDate, Type, Status)
        VALUES
            (@ApplicantID,
             CONCAT('You did not pass your driving exam on ',
                    CONVERT(VARCHAR(10), @ExamDate, 120),
                    '. Score: ', @Score,
                    '. Please re-book.'),
             GETDATE(),
             'Rejection',
             'Unread')
    END
```

```
END
GO
-- Test the trigger
SELECT * FROM NOTIFICATION WHERE ApplicantID = 3

UPDATE DRIVING_EXAM
SET Status = 'Passed', Score = 78.50
WHERE ExamID = 3

SELECT * FROM NOTIFICATION WHERE ApplicantID = 3
GO
```

```
1018  CREATE TRIGGER trg_auto_notify_exam_result
1019  ON DRIVING_EXAM
1020  AFTER UPDATE
1021  AS
1022  BEGIN
1023      DECLARE @ApplicantID INT
1024      DECLARE @Status      VARCHAR(10)
1025      DECLARE @ExamDate    DATE
1026      DECLARE @Score       DECIMAL(5,2)
1027
1028      -- Step 1: Get the updated row from INSERTED table
1029      SELECT @ApplicantID = ApplicantID,
1030             @Status      = Status,
1031             @ExamDate    = ExamDate,
1032             @Score       = Score
1033      FROM INSERTED
```

110 % 16 0 Ln: 1078, Ch

Results Messages

	NotifID	ApplicantID	Message	NotifDate	Type	Status
1	4	3	Your application is currently under review.	2026-02-05 08:00:00.000	Reminder	Unread
2	6	3	You PASSED your driving exam on 2026-03-01. Scor...	2026-09-26 21:49:14.470	Approval	Unread
3	7	3	You PASSED your driving exam on 2026-03-01. Scor...	2026-09-26 21:52:21.277	Approval	Unread

	NotifID	ApplicantID	Message	NotifDate	Type	Status
1	4	3	Your application is currently under review.	2026-02-05 08:00:00.000	Reminder	Unread
2	6	3	You PASSED your driving exam on 2026-03-01. Scor...	2026-09-26 21:49:14.470	Approval	Unread
3	7	3	You PASSED your driving exam on 2026-03-01. Scor...	2026-09-26 21:52:21.277	Approval	Unread
4	8	3	You PASSED your driving exam on 2026-03-01. Scor...	2026-09-26 21:52:47.617	Approval	Unread

	NotifID	ApplicantID	Message	NotifDate	Type	Status
1	4	3	Your application is currently under review.	2026-02-05 08:00:00.000	Reminder	Unread
2	6	3	You PASSED your driving exam on 2026-03-01. Scor...	2026-09-26 21:49:14.470	Approval	Unread
3	7	3	You PASSED your driving exam on 2026-03-01. Scor...	2026-09-26 21:52:21.277	Approval	Unread
4	8	3	You PASSED your driving exam on 2026-03-01. Scor...	2026-09-26 21:52:47.617	Approval	Unread

Trigger 2 - trg_check_exam_slot_capacity

Blocks an INSERT into EXAM_SLOT_ASSIGNMENT once the target EXAM_SLOT has reached Capacity, preventing overbooking.

```
CREATE TRIGGER trg_check_exam_slot_capacity
ON EXAM_SLOT_ASSIGNMENT
INSTEAD OF INSERT
AS
BEGIN
    DECLARE @SlotID   INT
    DECLARE @ExamID   INT
    DECLARE @Capacity INT
    DECLARE @Booked   INT

    SELECT @SlotID = SlotID,
           @ExamID = ExamID
    FROM INSERTED

    SELECT @Capacity = Capacity
    FROM EXAM_SLOT
    WHERE SlotID = @SlotID

    IF @Capacity IS NULL
    BEGIN
        PRINT 'ERROR: Exam slot not found.'
        RETURN
    END

    SELECT @Booked = COUNT(*)
    FROM EXAM_SLOT_ASSIGNMENT
    WHERE SlotID = @SlotID

    IF @Booked >= @Capacity
    BEGIN
        PRINT 'ERROR: This exam slot is full. Booking rejected.'
        RETURN
    END
    ELSE
    BEGIN
        INSERT INTO EXAM_SLOT_ASSIGNMENT (ExamID, SlotID)
        VALUES (@ExamID, @SlotID)

        PRINT 'SUCCESS: Exam slot booked successfully.'
    END
END
GO
-- Remove previous test booking if it exists
DELETE FROM EXAM_SLOT_ASSIGNMENT
WHERE ExamID = 4
  AND SlotID = 2;
GO

-- Make slot 2 full
UPDATE EXAM_SLOT
```

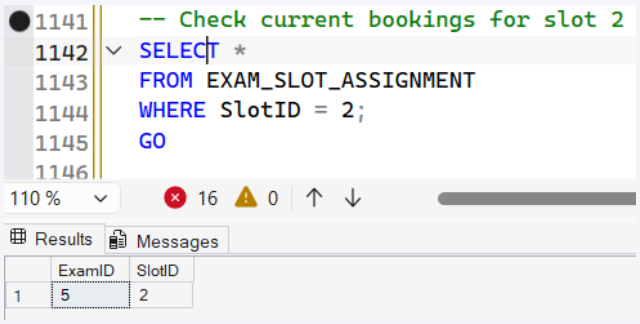
```
SET Capacity = 1
WHERE SlotID = 2;
GO

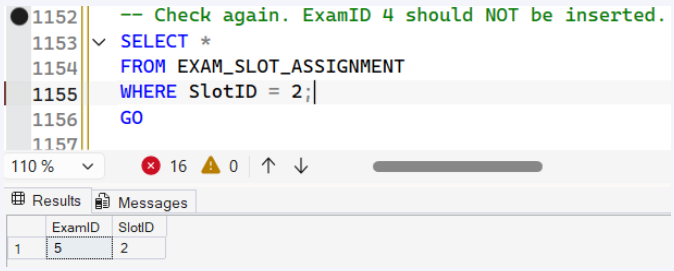
-- Check current bookings for slot 2
SELECT *
FROM EXAM_SLOT_ASSIGNMENT
WHERE SlotID = 2;
GO

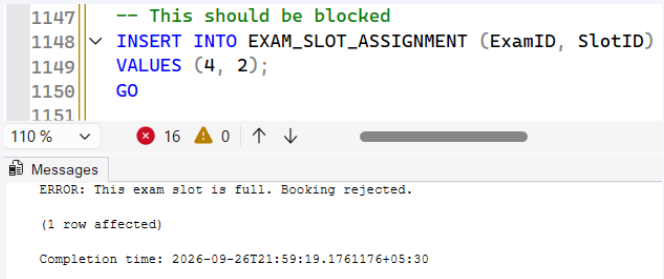
-- This should be blocked
INSERT INTO EXAM_SLOT_ASSIGNMENT (ExamID, SlotID)
VALUES (4, 2);
GO

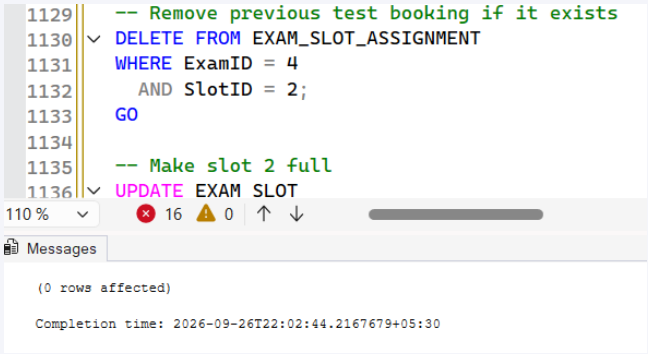
-- Check again. ExamID 4 should NOT be inserted.
SELECT *
FROM EXAM_SLOT_ASSIGNMENT
WHERE SlotID = 2;
GO

-- Reset capacity
UPDATE EXAM_SLOT
SET Capacity = 25
WHERE SlotID = 2;
GO
```









EER corrected diagram

Primary key underlines are not clear due to low picture quality. Advised by instructor to reupload.

